

ECSE 222, Digital Logic

VHDL 4:

Getting Started with Altera DE1-SoC Board and Sequential Statements

Fall 2023
University of McGill

Presented by:

Nara Yun (Student #: 261115268)

Karen Chen Lai (Student #: 261107674)

Instructors: Prof. Boris Vaisband

Teaching Assistant: Maninder Bir Singh Gulshan

Date: November 7th, 2023

Executive Summary

In this VHDL assignment, our group worked on designing and implementing several digital circuits on the Altera DE1-SoC board. The main objectives of this assignment were to create a BCD to 7-segment LED decoder, a wrapper circuit for a 4-bit BCD adder, and a 4-bit comparator.

First part consists of the design of a BCD to 7-Segment LED Decoder. We implemented a BCD to 7-segment LED decoder in VHDL files, which takes 4-bit BCD-formatted input and generates the appropriate 7-segment display output.

The second part is the design of a Wrapper Circuit. For the wrapper circuit, we designed a 4-bit BCD adder that takes two 4-bit BCD inputs (A and B) and displays the inputs as well as the result of the addition on the 7-segment LEDs. We used the binary-to-7-segment LED decoder for the display.

The third is creating a 4-Bit comparator. We implemented a 4-bit comparator that compares two 4-bit unsigned inputs (A and B). The comparator circuit determined whether A was greater than B+1, greater than or equal to B+1, less than B+1, less than or equal to B+1, or equal to B+1. It also detected overflow when B+1 required more than 4 bits.

We tested both the wrapper and comparator on the FPGA board and verified its functionality using LEDs and slider switches.

The last part is the critical Path Analysis. We conducted simulation tests to ensure the correct behavior of the circuits and provided simulation plots for different test cases.

Questions

1. Briefly explain your VHDL code implementation of all circuits.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity seven_segment_decoder is
6  port(code: in std_logic_vector (3 downto 0);
7  segments_out: out std_logic_vector (6 downto 0));
8  end seven_segment_decoder;
9
10 architecture decoder of seven_segment_decoder is
11 begin
12     WITH code SELECT
13     segments_out <=
14         "1000000" WHEN "0000", -- 0
15         "1111001" WHEN "0001", -- 1
16         "0100100" WHEN "0010", -- 2
17         "0110000" WHEN "0011", -- 3
18         "0011001" WHEN "0100", -- 4
19         "0010010" WHEN "0101", -- 5
20         "0000010" WHEN "0110", -- 6
21         "1111000" WHEN "0111", -- 7
22         "0000000" WHEN "1000", -- 8
23         "0010000" WHEN "1001", -- 9
24         "1111111" WHEN others;
25 end decoder;
```

Figure 1. Seven segment decoder (Converts a 4-bit BCD (Binary Coded Decimal) to a 7-segment display code.)

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity Karen_chen_comparator is
6  port ( A, B : in std_logic_vector(3 downto 0);
7        AgtBplusone : out std_logic;
8        AgteBplusone : out std_logic;
9        AltBplusone : out std_logic;
10       AlteBplusone : out std_logic;
11       AeqBplusone : out std_logic;
12       overflow : out std_logic);
13  end Karen_chen_comparator;
14
15  architecture comp of Karen_chen_comparator is
16  --5-bit to account for overflow
17  signal Bplusone: std_logic_vector(4 downto 0);
18
19  begin
20  process(A,B)
21  begin
22      AgtBplusone <= '0';
23      AgteBplusone <= '0';
24      AltBplusone <= '0';
25      AlteBplusone <= '0';
26      AeqBplusone <= '0';
27      overflow <= '0';
28
29      if to_integer(unsigned(B)) + 1 >= 16 then
30          overflow <= '1';
31      else
32
33          if to_integer(unsigned(B)) + 1 = to_integer(unsigned(A)) then
34              AeqBplusone <= '1';
35              AgteBplusone <= '1';
36              AltBplusone <= '1';
37
38          elsif to_integer(unsigned(B)) + 1 > to_integer(unsigned(A)) then
39              AltBplusone <= '1';
40              AlteBplusone <= '1';
41
42          else
43
44              AgtBplusone <= '1';
45              AgteBplusone <= '1';
46          end if;
47      end if;
48  end process;
49  end comp;
50
51 end comp;

```

Figure 2. 4-bit comparator

Figure 2 illustrates a component that is devised to carry out a comparison between two 4-bit numbers, A and B. The outputs generated by this comparison are AgtBplusOne, AeqBplusOne, AltBplusOne, and overflow. The output AgtBplusOne is set to 1 if A exceeds B+1; otherwise, it remains 0. Similarly, AeqBplusOne is asserted as 1 if A precisely matches B+1; if not, it's 0. AltBplusOne goes high (i.e., becomes 1) if A is smaller than B+1, and if that's not the case, it's set to 0. The overflow output is particularly for flagging scenarios where adding 1 to B would result in an overflow, which specifically happens when B has the maximum 4-bit value of 15.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity Karen_chen_comparator_tb is
6  end Karen_chen_comparator_tb;
7  architecture tb of Karen_chen_comparator_tb is
8
9  -- signals
10  signal A: std_logic_vector(3 downto 0);
11  signal B: std_logic_vector(3 downto 0);
12  signal AgtBplusone: std_logic;
13  signal AgteBplusone: std_logic;
14  signal AltBplusone: std_logic;
15  signal AlteBplusone: std_logic;
16  signal AeqBplusone: std_logic;
17  signal overflow: std_logic;
18
19  component Karen_chen_comparator
20  port ( A, B: in std_logic_vector(3 downto 0);
21        AgtBplusone: out std_logic;
22        AgteBplusone: out std_logic;
23        AltBplusone: out std_logic;
24        AlteBplusone: out std_logic;
25        AeqBplusone: out std_logic;
26        overflow: out std_logic);
27  end component;
28
29  begin
30  comparator : Karen_chen_comparator
31  port map (A, B, AgtBplusone, AgteBplusone, AltBplusone, AlteBplusone, AeqBplusone, overflow);
32
33  comparator_test: process
34  begin
35      A <= "0101";
36
37      for i in 0 to 16 loop
38          B <= std_logic_vector(to_unsigned(i, 4));
39          wait for 10 ns;
40      end loop;
41
42      wait;
43  end process comparator_test;
44
45  end tb;

```

Figure 3. 4-bit comparator testbench

Figure 3 is designed to test the Karen_Chen_comparator. It initializes A to "0101" (5 in decimal) and iteratively assigns all possible values to B from 0 to 15.

2. Provide the VHDL code that you wrote for the wrapper circuit.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity Karen_Chen_wrapper is
6  port ( A, B : in std_logic_vector(3 downto 0);
7        decoded_A, decoded_B : out std_logic_vector (6 downto 0);
8        decoded_AplusB : out std_logic_vector (13 downto 0));
9  end Karen_Chen_wrapper;
10
11 architecture wrapper of Karen_Chen_wrapper is
12     component seven_segment_decoder is
13     port(
14         code : in std_logic_vector(3 downto 0);
15         segments_out : out std_logic_vector(6 downto 0));
16     end component;
17
18     component Karen_Chen_bcd_adder is
19     port(
20         A, B : in std_logic_vector(3 downto 0);
21         S : out std_logic_vector(3 downto 0);
22         C : out std_logic);
23     end component;
24
25     signal AplusB : std_logic_vector(3 downto 0);
26     signal C : std_logic;
27
28 begin
29     -- 7-segments for A
30     SA: seven_segment_decoder
31     port map(code => A, segments_out => decoded_A);
32
33     -- 7-segments for B
34     SB: seven_segment_decoder
35     port map(code => B, segments_out => decoded_B);
36
37     -- 7-segments for A+B
38     BCD: Karen_Chen_bcd_adder
39     port map(A, B, AplusB, C);
40
41     -- Displaying A+B, the first 4-bits
42     SAplusB_6to0: seven_segment_decoder
43     port map(code => AplusB, segments_out => (decoded_AplusB(6 downto 0)));
44
45     -- Displaying A+B, the last 4-bits
46     SAplusB_13to7: seven_segment_decoder
47     port map(code => ("000" & C), segments_out => (decoded_AplusB(13 downto 7)));
48 end;
49
50

```

Figure 4. Wrapper circuit

The codes for wrapper are illustrated in Fig.4, it uses two instances of the seven-segment decoder: one for A and one for B. There's also a mapping to display A+B on a seven-segment display, divided into two sections: the first 4 bits (most significant) and the remaining bits.

3. Show a representative photo of the board displaying the result of the addition of A and B, where A is the last (rightmost) digit of the McGill ID number of one of the students in your group and B is the last (rightmost) digit of the McGill ID number of the other student in the group (if there are three members in your group, choose two IDs and state which were chosen).

Clearly indicate which 7-segment LEDs/sliding switches are assigned to which inputs/outputs of the circuit on the photo.

A: $4_{10} = (0100)_2$

B: $8_{10} = (1000)_2$

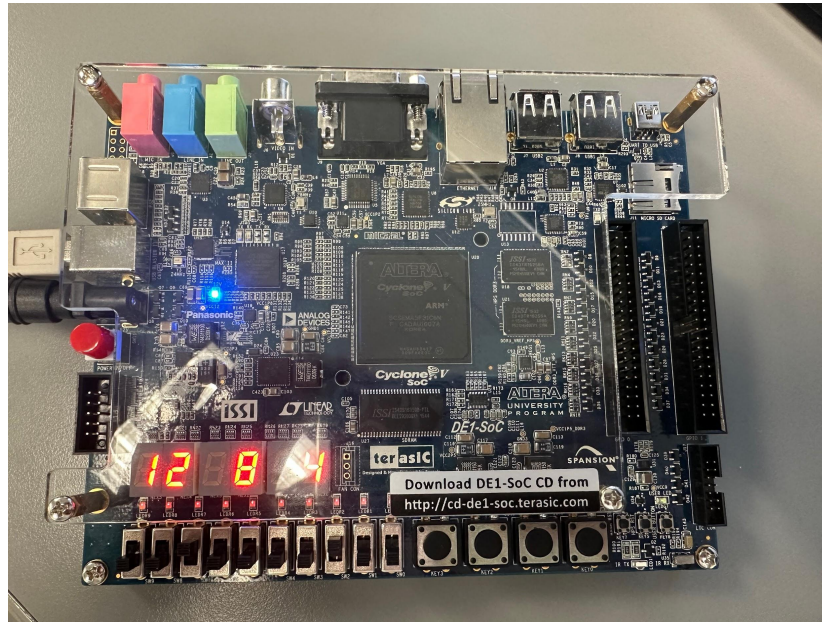


Figure 5. BCD to 7-segment decoder on DE1-SoC board

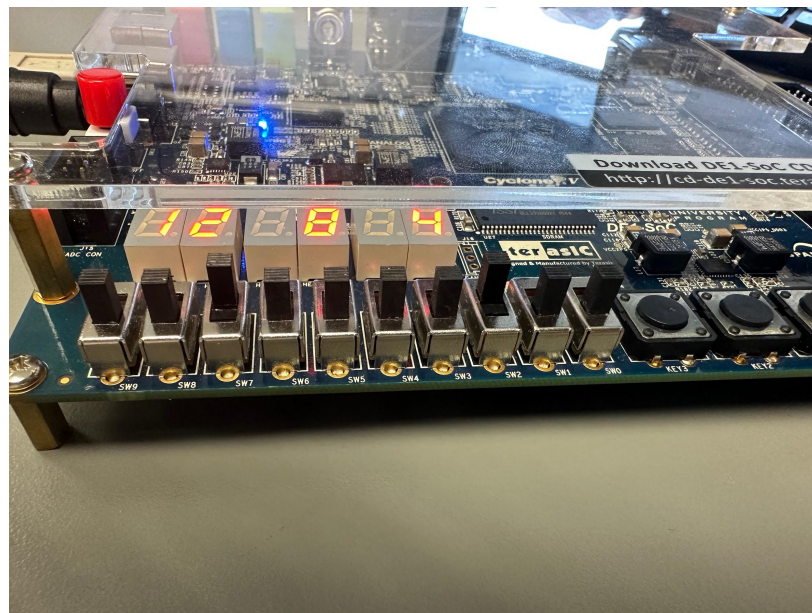


Figure 6. Slide switches on DE1-SoC board

Fig. 5 displays the number A at leftmost, number B at the center and the sum of A and B at leftmost. Fig. 6 shows the slide switches utilized to insert our inputs. The SW2 is on for the input A, which is $(0100)_2$, and the SW7 is on for the input B, $(1000)_2$.

4. Report the number of pins and logic modules used to fit your designs on the FPGA board.

Flow Status	Successful - Thu Nov 02 15:07:54 2023
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Standard Edition
Revision Name	VHDL4_Karen_Chen
Top-level Entity Name	Karen_Chen_wrapper
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	19 / 32,070 (< 1 %)
Total registers	0
Total pins	36 / 457 (8 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 7. Compilation report of wrapper

Flow Status	Successful - Thu Nov 02 15:13:36 2023
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Standard Edition
Revision Name	VHDL4_Karen_Chen
Top-level Entity Name	Karen_Chen_comparator
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	7 / 32,070 (< 1 %)
Total registers	0
Total pins	14 / 457 (3 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 8. Compilation report of 4-bit comparator

5. Given that $A = 5_{10}$, provide a separate simulation plot that demonstrates all possible cases for the 4-bit comparator, including a separate plot for the case where overflow occurs. A total of four plots should be included. Explain each plot and mark all inputs and outputs clearly.

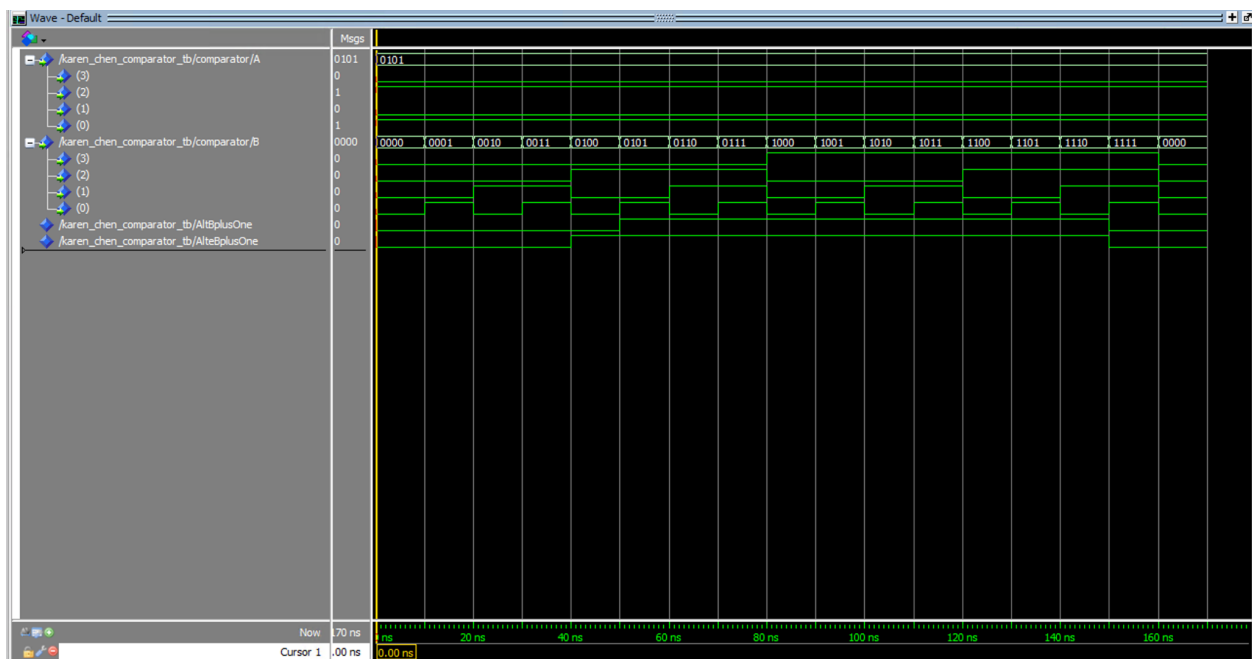


Figure 9. Simulation plot for $A < (B+1)$ and $A \leq (B+1)$

The waveform in Fig.9, the constant input A is set to '1010' in binary, representing the number 10 in decimal, while the variable input, B, iterates through all 16 possible 4-bit combinations, spanning from 0 to 15 in decimal. The output signal AgtBplusOne stays high when B's values are less than 9, suggesting that A is indeed greater than B+1 for these cases. As B increments, notably when reaching '1001' (or 9 in decimal), this signal drops to low, indicating that 10 is no longer greater than 9+1. Concurrently, the output AltBplusOne initiates a high signal when B's value is '1010' (or 10 in decimal), denoting that A (with a value of 10) is now less than B+1 for subsequent values of B. Notably absent is the output for AeqBplusOne, which would ideally pulse high when B is 9, indicating the condition where A equals B+1.

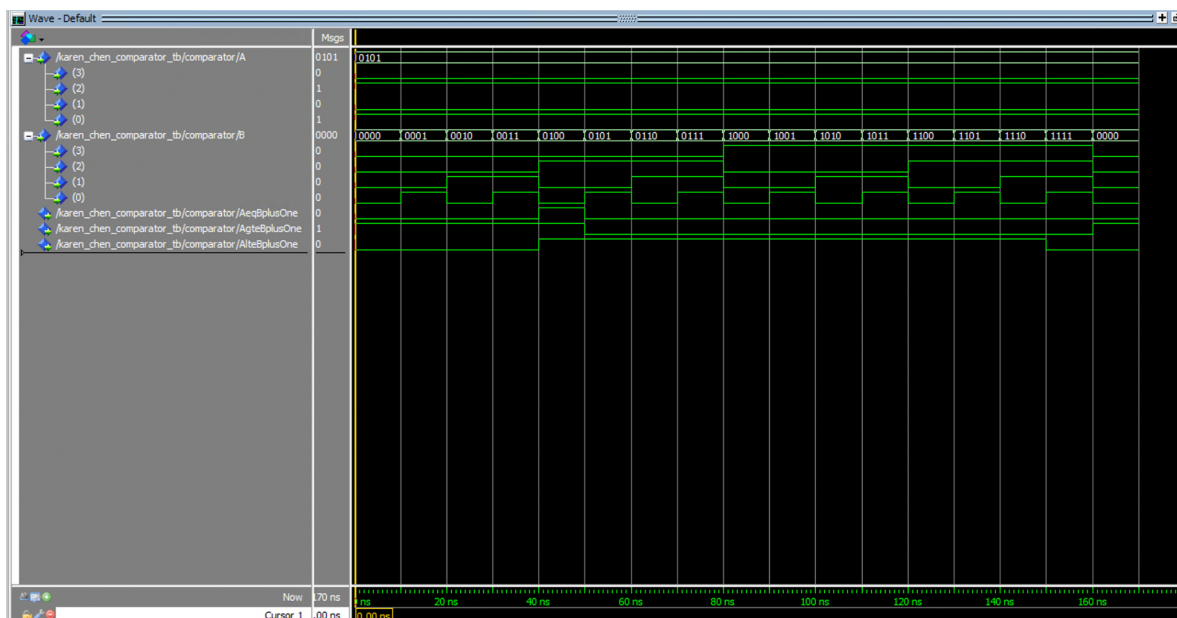


Figure 10. Simulation plot for $A > (B+1)$ and $A \geq (B+1)$

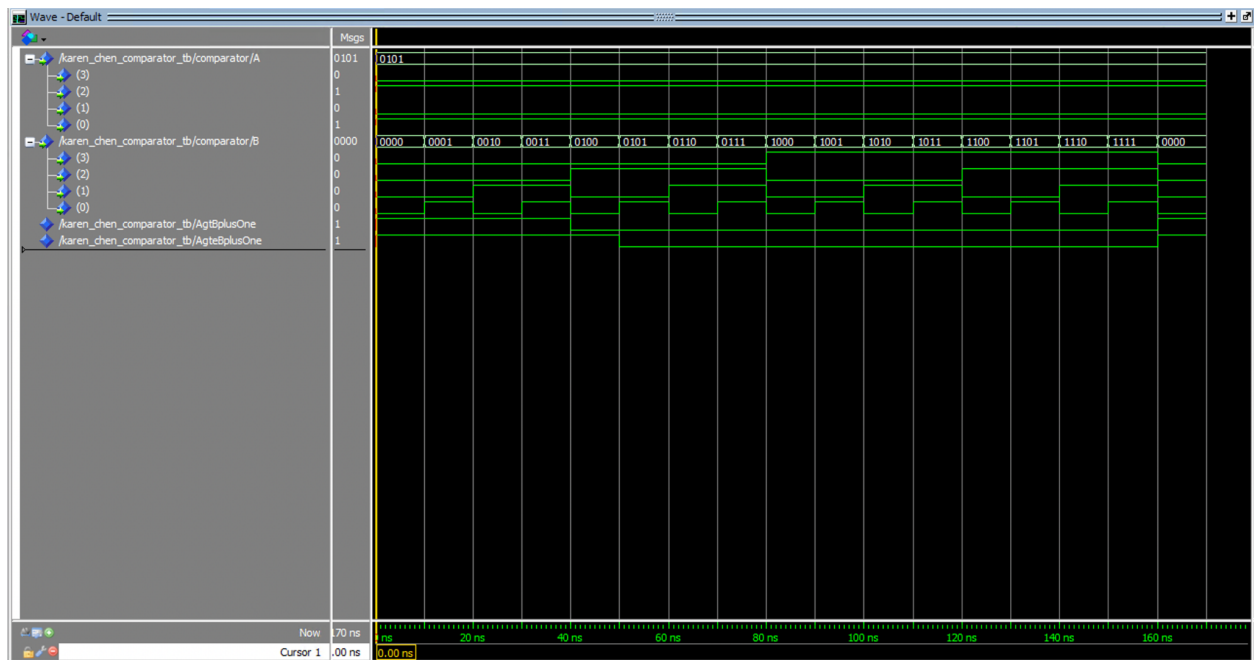


Figure 11. Simulation plot for $A \geq (B+1)$ and $A \leq (B+1)$ and $A = (B+1)$

The waveform in Figure 10 depicts a the case of equalness. The output, AgtBplusOne, signifies that A's value is greater than B + 1 for B values from 0 to 8. However, when B equals 9, this relation no longer holds. Another output, AgtEplusOne, becomes active when B reaches 10, indicating that from this point, A is greater than or equal to B + 1.

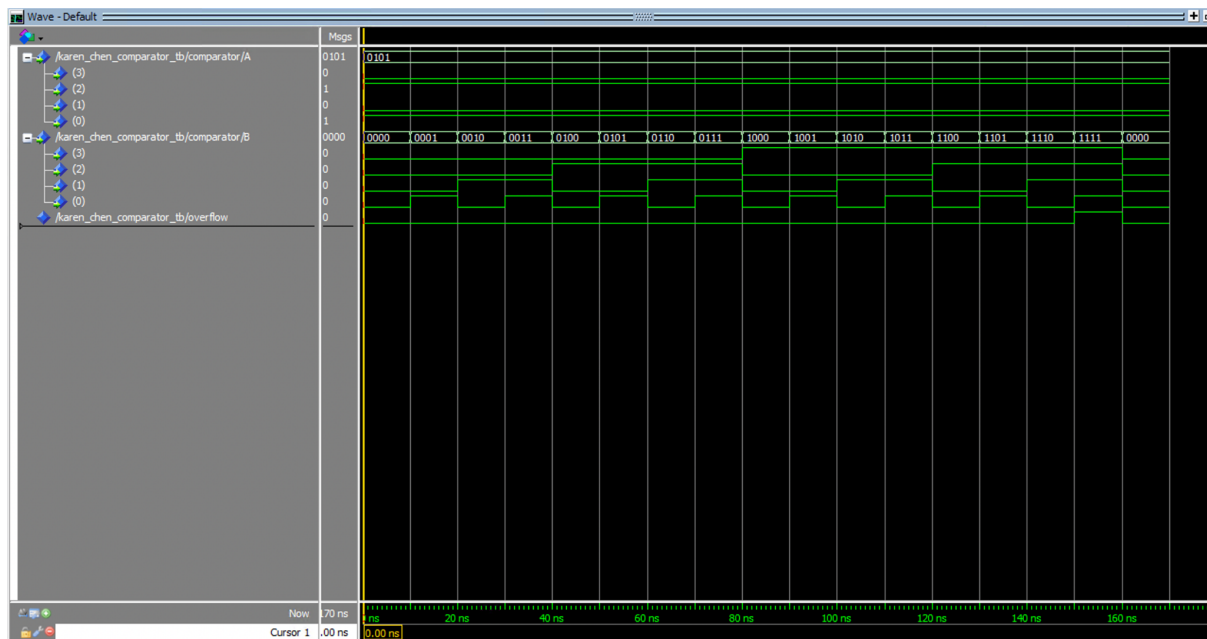


Figure 12. Simulation plot for overflow

In Figure 11, the 'AeqBplusOne' signal is activated when B has a value of '1001' or 9 in decimal, since $9 + 1$ is 10, the exact value of A. The 'AgtEplusOne' signal turns on from the moment B equals '1001' and remains active for the rest of the simulation, highlighting that the value of A is consistently greater than or equal to B+1 from this juncture. Conversely, the

'AlteBplusOne' signal persists in its active state until B hits '1000', suggesting that for all B values up to 8, A is always less than or equal to B+1.

In figure 12, the overflow signal in this simulation remains consistently low, indicating that no overflow conditions arise throughout this simulation. The overflow signal is activated only when B is equal to '1111' since the range of 4-bit number is from 0 to 15 only.

6. Perform timing analysis (slow 1,100 mV model) of the 4-bit comparator and find the critical path(s) of the circuit. What is the delay of the critical path(s)?

Top Failing Paths [hide details]				
	Slack	From	To	Recommendations
1	-6.364	B[0]	AgteBplusOne	Report recommendations for this path
2	-6.352	B[0]	AgteBplusOne	Report recommendations for this path
3	-6.281	B[1]	AgteBplusOne	Report recommendations for this path
4	-6.259	B[2]	AgteBplusOne	Report recommendations for this path
5	-6.258	A[1]	AgteBplusOne	Report recommendations for this path

Figure 13. Top failing paths

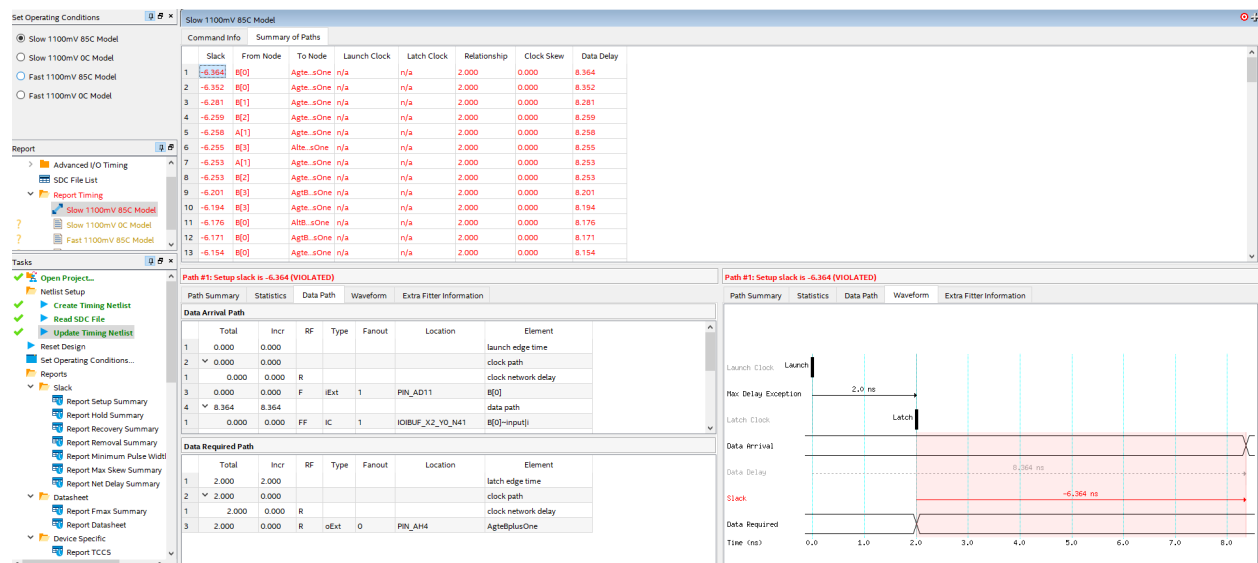


Figure 14. Critical path of the 4-bit comparator

As shown in Fig. 14, the delay of the critical path of the 4-bit comparator is 8.364.

Explanations of the Results

The BCD to 7-segment LED decoder successfully converted the 4-bit BCD-formatted input into the appropriate 7-segment display output. The VHDL code for the decoder utilized a

"WITH...SELECT" structure to map BCD values to the corresponding 7-segment display codes. When the input BCD value was changed using the sliding switches on the DE1-SoC board, the decoder correctly displayed the corresponding digits on the 7-segment LED displays. This demonstrates the effectiveness of the decoder in converting binary-coded decimal values into a visual representation on the LEDs.

The wrapper circuit, which incorporated a 4-bit BCD adder and the BCD to 7-segment LED decoder, displayed the inputs (A and B) and the result of the addition in BCD format on the 7-segment LED displays. When we set the input values A and B using the sliding switches, the wrapper circuit processed the input and displayed the results accurately.

The 4-bit comparator was designed to compare two 4-bit unsigned inputs (A and B) and determine various conditions such as $A > B$, $A \geq B$, $A < B$, $A \leq B$, and $A = B$. Additionally, it could detect overflow when B+1 required more than 4 bits. Testing the comparator using slider switches for input values and LEDs for output signals, we observed the correct behavior for different test cases. This demonstrated that the comparator accurately assessed the relationship between A and B, including the condition where an overflow occurred due to the limitations of 4-bit arithmetic.

Simulation and Timing Analysis:

During the simulation, we verified that the circuits behaved as expected for various input combinations. We provided simulation plots to illustrate different test cases, highlighting the results for each condition. In the timing analysis, we considered the slow 1,100 mV model and identified the critical path(s) of the 4-bit comparator circuit. This analysis helped us understand the potential delays and performance characteristics of the design, which can be crucial in real-time applications.

Conclusions

Through this experiment, we:

- Improved our understanding on to BCD through creating the 7-segment LED decoder.
- Created a wrapper circuit for a 4-bit BCD adder.
- Built a 4-bit comparator through describing it using IF-THEN-ELSIF statements.
- Tested digital circuits on Altera DE1-SoC board.
- Learnt how to manage sliding switches on the Altera DE1-SoC board to specify our inputs.
- Practiced using Quartus Timing Analyzer tools to find the critical path.